Graphwise Stack — Terraform module notes

Maintainer: Kent Stroker

This document covers how the Terraform module is structured, what each file does, and how `user-data.sh.tpl` bootstraps the EC2 instance. For the full step-by-step deploy walkthrough see DEPLOYMENT_GUIDE.md; for the operator's concise deploy steps see STACK-BUILDER.md.

What this module provisions

`infra/terraform-<stack>/` is a self-contained Terraform module that creates five AWS resources:

Resource	Details
`aws_security_group.stack`	Inbound: SSH, HTTP (port 80), and HTTPS (port 443) all restricted to `admin_cidr`. Outbound: all. `ignore_changes = [ingress]` so manual SG additions (EC2 Instance Connect) survive future applies. LE cert issuance uses DNS-01 exclusively — no inbound port required.
`aws_instance.stack`	EC2 instance (default `r6g.2xlarge`, AL2023 ARM64, 300 GiB encrypted gp3). `ignore_changes = [ami, user_data_base64]` prevents AMI-lookup drift and user-data edits from forcing a rebuild.
`aws_iam_role` + `aws_iam_instance_profile`	EC2 instance role with a single inline policy (`graphwise-stack-route53`) granting `route53:ChangeResourceRecordSets` + `route53:ListResourceRecordSets` scoped to the hosted zone ARN built from `route53_zone_id`. cert-manager uses this role (via IMDSv2) for DNS-01 wildcard cert issuance.
`aws_eip_association` **or** `aws_eip`	If `existing_eip_allocation_id` is set: associates the pre-allocated EIP (EIP itself lives outside Terraform; destroy only detaches). If the var is empty: creates a fresh EIP that is released on destroy. Always use the pre-allocated path — a fresh EIP means re-doing DNS after every rebuild.
`random_id.n8n_key`	Generates the 32-byte `n8n_encryption_key` once. Stored in Terraform state; never regenerated unless `terraform destroy` +apply. Changing it makes every saved n8n credential unreadable.

What the module does **not** manage: DNS records, license files, Kubernetes objects, Helm releases, Let's Encrypt certs, IAM user creation (Terraform or Bedrock users — done by root/IAM-admin as a one-time human step).

File map

```
infra/terraform-<stack>/
├── versions.tf               Terraform + AWS provider version pins
├── variables.tf              All input variables with validation rules + defaults
├── main.tf                   SG, EC2, IAM role/profile, EIP logic
├── outputs.tf                elastic_ip, ssh, ami_id, route53_dns_records, expected_urls
├── user-data.sh.tpl          Cloud-init bootstrap script (runs as root on first boot)
├── terraform.tfvars.example  Copy → terraform.tfvars, fill REQUIRED block
├── terraform.tfvars          [gitignored] — per-deployment values
└── scripts/                  Laptop-side helper scripts (not part of the Terraform module)
    ├── check-prereqs.sh      macOS/Windows prereq checker
    ├── push-config.sh        Restore operator files to EC2 after a rebuild
    ├── pull-config.sh        Snapshot operator files from EC2 before destroy
    ├── manage-stacks.sh      Add/list/remove stack SSH entries in ~/.zprofile
    └── stack-scp.sh          scp wrapper auto-filling key+host from ~/.zprofile
```

`terraform.tfvars` is the only file operators edit. `variables.tf` is module source — never edit it for customization (see below).

Key variables

Required variables (no default — apply fails without them):

Variable	Notes
`region`	AWS region. Must be Bedrock-enabled or `graphrag-secrets.awsCredentials.region` must point at one.
`base_domain`	Parent domain in a Route 53 hosted zone you own. PSE SEs use `gw-pse.com`.
`route53_zone_id`	Hosted zone ID for `base_domain`. Format: `Z<UPPERCASE-ALPHANUM>`. **Validation regex `^Z[A-Z0-9]+$` accepts well-formed but wrong IDs.** Always verify with `aws route53 list-hosted-zones`. A wrong ID creates an IAM policy scoped to a nonexistent zone ARN → cert-manager gets `AccessDenied` on DNS-01 → wildcard cert never issues.
`le_email`	Let's Encrypt ACME contact. Written to `/etc/profile.d/graphwise.sh` so `cluster-bootstrap.sh` picks it up automatically.
`subdomain`	Subdomain under `base_domain`. Drives the apex hostname `<sub>.<base>` and all per-app subdomains `<app>.<sub>.<base>`.
`creator`	Attribution tag on every AWS resource. Required (apply fails if empty) so nothing lands in a shared account anonymously.
`key_pair_name`	Name of a pre-existing EC2 key pair in the target region.
`admin_cidr`	Your current public IPv4 + `/32`. Restricts SSH inbound. Never `0.0.0.0/0`.
`availability_zone`	Must be in `region`.
`existing_eip_allocation_id`	Pre-allocated EIP allocation ID (`eipalloc-...`). Strongly recommended — keeps the IP stable across destroy/apply cycles so DNS stays valid.

Notable optional variables:

Variable	Default	Notes
`instance_type`	`r6g.2xlarge`	Graviton ARM64. `r6g.xlarge` works for lightweight demos (JVM heaps tighter).
`root_volume_gb`	`300`	gp3, encrypted.
`ami_override`	`""`	Pin the AMI after first apply: `terraform output -raw ami_id` → paste here → `terraform plan` must show "No changes". Prevents spurious force-replace from AMI lookup drift.
`github_repo_url`	upstream public URL	Repo cloned onto EC2 by cloud-init. Override for feature branches.
`github_branch`	`"main"`	Branch cloned. Override only when testing pre-merge chart changes.

The Safety rule — never unscoped `terraform apply` post-provision

`data "aws_ami" "al2023_arm64"` uses `most_recent = true`. Every plan re-resolves the latest published AL2023 ARM64 AMI. If the resolved ID differs from state, Terraform marks `aws_instance.stack` for **force-replace** (destroy + recreate — all data gone) even for a trivial change like an SG tag edit.

Two-layer protection already in place:

1. `lifecycle.ignore_changes = [ami]` on `aws_instance.stack` — once provisioned, Terraform never marks it for replacement on AMI grounds, regardless of what the data source resolves. This is the belt.
2. `ami_override` in `terraform.tfvars` — explicitly pins the AMI ID at the lookup site, keeping `terraform plan` output clean. Set this after first apply. This is the braces.

Safer paths for common edits post-provision:

What to change	Safer path
SSH/HTTP/HTTPS source CIDR (`admin_cidr`)	AWS Console → EC2 → Security Groups → edit all three rules directly. Update `terraform.tfvars` for documentation only. For existing stacks: `terraform apply` won't touch SG rules due to `ignore_changes = [ingress]` — Console edit is required.
EC2 Instance Connect SG rule	Console-only manual addition. Survives future applies due to `ignore_changes = [ingress]` .
Extra resource tags	AWS Console → Tags tab → edit. Update `terraform.tfvars` to match.
`instance_type` / `root_volume_gb`	Force-replace by design. Snapshot EBS first.
Any other targeted change	`terraform plan -target=<resource>` — read the output fully before applying.

`user-data.sh.tpl` — cloud-init bootstrap deep dive

This file is the EC2 first-boot script. Terraform renders it via `templatefile()`, base64-encodes the result, and passes it as `user_data` on `aws_instance.stack`. AWS injects it into the instance at first boot; cloud-init runs it as root once. Output goes to `/var/log/bootstrap.log` and the system journal (`logger -t bootstrap`).

Template rendering and escaping

`user-data.sh.tpl` is a Terraform template, not a raw shell script. Two escaping rules:

Construct	Meaning
`${var_name}`	Terraform substitution — replaced with the variable's value at render time.
`$${SHELL_VAR}`	Escaped Terraform brace — renders as `${SHELL_VAR}` in the final script so the shell expands it at runtime, not Terraform at render time.

All Terraform-substituted variables in the template:

Template variable	Source	Used for
`${github_repo_url}`	`var.github_repo_url`	`git clone` target
`${github_branch}`	`var.github_branch`	Branch to check out
`${hostname_fqdn}`	`"${var.subdomain}.${var.base_domain}"`	Written to `/etc/profile.d/graphwise.sh` as `GRAPHWISE_APEX`; consumed by `cluster-bootstrap.sh` and `render-values.sh`
`${route53_zone_id}`	`var.route53_zone_id`	Written to `/etc/profile.d/graphwise.sh` as `ROUTE53_ZONE_ID`; consumed by `cluster-bootstrap.sh` to create the cert-manager ClusterIssuer
`${aws_region}`	`var.region`	Written to `/etc/profile.d/graphwise.sh` as `AWS_REGION`
`${le_email}`	`var.le_email`	Written to `/etc/profile.d/graphwise.sh` as `LE_EMAIL`
`${n8n_encryption_key}`	`random_id.n8n_key.b64_std`	Written into `~/graphwise-secrets.yaml` under `n8nEncryption.key`
`${graphwise_secrets_b64}`	`filebase64(local.secrets_file)`	Operator's pre-filled `graphwise-secrets.yaml`, inlined as base64. If absent, renders as empty string and a placeholder secrets file is written.
`${n8n_txt_b64}`	`filebase64(local.n8n_txt_file)`	`n8n.txt` (AWS credentials for n8n)
`${poolparty_key_b64}`	`filebase64(...)`	PoolParty license key
`${graphdb_license_b64}`	`filebase64(...)`	GraphDB EE license
`${uv_license_key_b64}`	`filebase64(...)`	UnifiedViews license key

The 16 KB limit. AWS user-data has a 16 KB cap after base64-encoding. Operator files (secrets, licenses) are the main pressure on this limit — each base64-encodes to ~4/3 its raw size. The `_wb64()` helper in the template decodes a base64-var to a file in one line, keeping the template body compact. If you add more optional files, watch the rendered size: `wc -c user-data-rendered.sh` (rendered by `terraform plan` logs, or `terraform output -raw user_data_b64 | base64 -d | wc -c`).

What the bootstrap script does, step by step

1. **OS patches + packages** — `dnf upgrade -y --refresh` then installs Docker, git, jq, bind-utils, conntrack-tools, ethtool, socat, iproute, httpd-tools (for `htpasswd`), tar, gzip, ca-certificates, rsync, python3, python3-pip.
2. **sshd hardening** — writes `/etc/ssh/sshd_config.d/10-graphwise.conf`: `internal-sftp` subsystem (survives openssh upgrades), `MaxStartups 100:30:200`, `LoginGraceTime 30`. Restarts sshd.
3. **Kernel + cgroup settings** — `/etc/sysctl.d/99-kind.conf` sets `net.ipv4.ip_forward=1` (required for pod networking) and raises `fs.inotify.max_user_watches` / `fs.inotify.max_user_instances` (kubelet file-watching limits hit on large deploys).
4. **Docker** — `systemctl enable --now docker`, adds `ec2-user` to the `docker` group so KIND and kubectl run without `sudo`.
5. **Tool installs** — pinned versions of `kind`, `kubectl`, `helm` downloaded to `/usr/local/bin/`. Versions are constants near the top of the template (`KIND_VERSION`, `KUBECTL_VERSION`, `HELM_VERSION`) — bump deliberately and re-test the full flow.
6. **`/etc/profile.d/graphwise.sh`** — written from Terraform-substituted vars. Exports `GRAPHWISE_APEX`, `ROUTE53_ZONE_ID`, `AWS_REGION`, `LE_EMAIL`. `cluster-bootstrap.sh` sources this file at startup so operators never have to manually export these variables.
7. **`graphwise-cluster-resume.service`** — systemd `oneshot` unit that runs `~/gsb/scripts/cluster-resume.sh --if-exists` as `ec2-user` after Docker starts. Enabled at first boot; fires on every subsequent EC2 start so the KIND cluster comes back without operator action. The `--if-exists` flag makes it a no-op on first boot (no cluster yet).
8. **`/etc/profile.d/graphwise-hint.sh`** — interactive login hint. Shows "cloud-init still running" until `/var/lib/cloud/graphwise-bootstrap-complete` exists; shows "KIND cluster resuming..." while `graphwise-cluster-resume.service` is activating. Silent in the steady state.
9. **kubeconfig + aliases** — appended to `~ec2-user/.bashrc`: `KUBECONFIG`, `alias k=kubectl`, `alias kga='kubectl get all --all-namespaces'`.
10. **Clone repo + create KIND cluster** — runs as `ec2-user` via `sudo -u ec2-user -i bash <<INNER`. The login-shell form (`-i`) forces a fresh group lookup so the `docker` group membership is live before `kind create cluster`. Idempotent: skips clone if `~/gsb` exists, skips cluster create if `graphwise` cluster exists.
11. **Workflow DB seed (no longer shipped in the repo)** — earlier builds expanded an `n8n-pg-dumpall-*.sql.tar.gz` from the repo here. That seed is no longer shipped in the repo/clone at all. The workflow

DB seed now lives only on the EC2 home root as `$HOME/workflows-pg-dumpall-<date>-v<N>.sql` — scp'd up by the operator or produced on the box by `create-workflows-dumpall.sh` — and `restore-workflows-dumpall.sh` loads the NEWEST one (no-op if none is present).

12. **pip3 packages** — `pip3 install -r ~/gsb/requirements.txt` system-wide (available to all scripts on the host — e.g., the Python used by `extract-poolparty-realm.sh` for YAML manipulation).
13. `~/graphwise-secrets.yaml` — if `${graphwise_secrets_b64}` is non-empty (operator pre-filled their secrets file and it was present when `terraform apply` ran), that file is decoded verbatim. Otherwise a placeholder template is written with all fields empty except `n8nEncryption.key` (already filled from Terraform). File is owned `ec2-user`, mode `600`.
14. `~/staging-data/` — landing pad for ingest uploads from the operator's laptop (`rsync -azP ... :~/staging-data/`). Created here; the KIND `extraMounts` and Kubernetes PV/PVC wiring are in the chart and cluster config.
15. **Optional operator files** — `_wb64()` decodes each base64-var to a destination file if non-empty, skipping silently if empty. Writes: `~/n8n.txt` (AWS creds for n8n), `~/gsb/files/licenses/{poolparty.key,graphdb.license,uv-license.key}`.
16. **Optional fully-automated deploy** — commented-out block near the end of the template calls `~/gsb/scripts/deploy-stack.sh` non-interactively. Uncomment only when all operator files are present in the Terraform folder so they get inlined above. Default is manual — operators SSH in and run `deploy-stack.sh` themselves.
17. **Bootstrap-complete sentinel** — `touch /var/lib/cloud/graphwise-bootstrap-complete` silences the login hint and marks the end of first-boot. Log line: `=== Bootstrap complete at <timestamp> ===`.

Don't edit `variables.tf` for customization

`variables.tf` is module source code. Every per-deployment knob belongs in `terraform.tfvars`. Editing `variables.tf` couples your deployment to your fork (can't pull upstream updates cleanly), hides values from anyone reading `terraform.tfvars`, and drifts from `terraform.tfvars.example`. The shipped example already lists all commonly-edited variables in its REQUIRED section — use it.

What to commit vs. ignore

Commit	Ignore
All `.tf` files	`terraform.tfvars` (real values including `admin_cidr`)
`terraform.tfvars.example`	`.terraform/` (provider binaries)
`user-data.sh.tpl`	`terraform.tfstate` / `*.tfstate.backup`
`.terraform.lock.hcl` (after `terraform init`)	`*.tfplan`

`.gitignore` at the repo root already covers all of these.

Troubleshooting — Terraform-layer issues

`InvalidAMIID.NotFound` during plan

The AMI data source didn't match. Check the region has AL2023 ARM64 images:

```
aws ec2 describe-images --owners amazon \
    --filters "Name=name,Values=al2023-ami-*-arm64" \
    --query 'Images[*].[Name,ImageId]' --output table --region <region>
```

If empty, pick a different region or try `terraform init` to refresh the provider.

`UnauthorizedOperation` on `RunInstances`

The `terraform-demo` IAM user is missing `ec2:RunInstances`. Attach `AmazonEC2FullAccess` (or the scoped custom policy from SETUP.md §4a) and retry.

`terraform apply` wants to replace `aws_instance.stack` on AMI change

The `lifecycle.ignore_changes = [ami]` block prevents this on an existing instance. If you see a force-replace on `ami`, the instance hasn't been provisioned yet (state is empty) — this is normal on first apply. After first apply, always run `terraform output -raw ami_id` and set `ami_override` in `terraform.tfvars`.

`terraform apply` wants to replace `aws_instance.stack` on `user_data_base64` change

Also covered by `lifecycle.ignore_changes`. User-data changes after first boot must be applied manually via SSH. To intentionally re-bootstrap: `terraform taint aws_instance.stack` + `terraform apply` (destroys and recreates — all data gone). Not normally needed.

Bootstrap script fails partway through

SSH in as `ec2-user`, read `/var/log/bootstrap.log`. The script runs under `set -e` so first error is final. Most common causes: - Transient `dnf` mirror failure — rerun the failing install line, then step through the remainder manually (script is largely idempotent). - Docker group not picked up — symptom: `Got permission denied … Docker daemon socket`. The `sudo -u ec2-user -i` login-shell form should handle this; if it didn't, run the KIND step manually as `ec2-user` after the bootstrap and file a bug.

cert-manager `AccessDenied` on Route 53 (DNS-01 challenge)

`route53_zone_id` in `terraform.tfvars` is wrong. The validation regex accepts any `Z[A-Z0-9]+` string including stale IDs. The IAM policy is created against the wrong zone ARN, so cert-manager's `ChangeResourceRecordSets` on the real zone fails. Immediate fix (no EC2 rebuild): `aws iam put-role-policy` with the corrected zone ARN. Then fix `terraform.tfvars` for consistency. See CLAUDE.md resolved bug catalog for the full diagnostic procedure.